Title: Bayesian neural network with autoencoder for model-based description of α-particle preformation factor
Manuscript ID: CN10966

Response to the reviewer's comments and suggestions

We sincerely appreciate your constructive comments and suggestions, which are useful and have helped us to improve the quality of our work significantly. Please accept our renewed thanks for your thoughtful feedback.

General comments:

1) The introduction lacks precision; the authors should more clearly define the specific contributions of this work. Recent literature includes studies utilizing encoder-decoder strategies (e.g., Ref [50] in the manuscript). Is the novel result the use of an encoder-decoder strategy combined with the CPT-LSM approach, or is the emphasis on the BNN approach, or perhaps a combination of encoder-decoder and BNN? In works introducing multiple components, it is advisable to address the influence of each individually or consecutively. For instance, one might first assess the CPT-LSM approach, then incorporate the influence of the encoder-decoder, and finally evaluate the effect of the BNN-auto approach. In the current manuscript, everything is implemented together, making it difficult to visualize the individual contribution of each part.

Response: Thank you very much for your constructive suggestions. Based on your advice, we have made targeted revisions and additions to the introduction of the manuscript, clearly distinguishing the roles and contributions of the three core elements (cosh potential model, autoencoder, Bayesian neural network) in this work, and citing the ablation experiments in the main text (S-BNN, KL-off) to illustrate the impact of each part one by one. These modifications have made the introduction of the work's innovative points more clear and precise. Once again, thank you for your guidance.

2) The RMS values obtained with the CPT-BNN-Auto ($\sigma_{RMS} = 0.346$) are only slightly lower than previous RF results (0.351) and other works. This improvement does not significantly outperform existing methods as stated. Consequently, the claim that these results clearly "outperform" previous works is insufficient. The authors appear to be overstating and exaggerating their claims on pages 15 to 24. Furthermore, the authors should only claim what they have demonstrated, providing evidence before making assertions rather than the reverse (for further detail, see specific comment 3 below).

Response: We sincerely thank you for this insightful comment. We fully acknowledge that the RMS improvement of the BNN-Auto (0.346) over the RF baseline (0.351) is indeed marginal, and our original wording such as "significantly outperforms"d overstated the actual advantage. To address this, we have thoroughly revised the relevant sections from pages 15 to 24 in the manuscript. All exaggerated claims have been replaced with more measured language: for instance, "significantly outperforms" has been changed to "achieves competitive performance" or "shows a slight improvement," and phrases like "strongly demonstrates" have been softened to "indicates". Moreover, we have added a brief clarification that while

the RMS difference over RF is modest, the main contributions of the BNN-Auto lie in its Bayesian uncertainty quantification, improved robustness on validation sets, and the ability to capture physically meaningful patterns such as odd-even staggering and shell effects-aspects that are not directly available from RF predictions. These revisions ensure that every claim is properly supported by the evidence presented, and we greatly appreciate the reviewer's rigorous evaluation in helping us improve the precision of our work.

3) Reproducibility concerns: The manuscript lacks a data availability section, which is necessary to verify the code implementation and database. Providing a GitHub link to the database and code is essential for reproducibility. Scientific progress relies on others being able to utilize results to implement new ideas.

Response: Thank you for your valuable suggestions. We fully understand and agree with the importance of open science and reproducible research, which is crucial for promoting the development of the field. We have made the complete code repository and trained model weights used in this work publicly available on GitHub (https://github.com/Lin-lab-code/BNN) to ensure that other researchers can fully reproduce our results and conduct further studies.

4) Computational efficiency is mentioned multiple times, yet specific computational times are not provided. The authors should compare, for example, the cost of CPT-LSM versus BNN-Auto using the same computational resources.

Response: Thank you for your meticulous review and the valuable questions you raised. The BNN-Auto model of this work is trained based on a dataset of 535 nuclides (with 8 input features). All deep learning experiments are conducted on a workstation equipped with an NVIDIA GeForce RTX 3090 (24GB) GPU, an Intel Xeon Platinum 8272CL CPU, and approximately 376GB of memory. For the final version of the BNN-Auto network in the revised manuscript, a complete 10-fold cross-validation training and validation are performed on a single RTX 3090, and the actual total time consumed is approximately 436.7 seconds (about 7.3 minutes), with an average training time per fold being approximately 43.6 ± 11.5 seconds. In contrast, the computational cost during the prediction stage is very low: a single Monte Carlo posterior prediction with $T = 20$ for all 535 nuclides only takes about 0.05 seconds. Regarding the comparison with CPT-LSM, it should be noted that the calculation paradigms of the two are different. CPT-LSM belongs to a parametric method based on physical models and least squares fitting, with its parameter scale much smaller than that of neural networks and not involving multiple rounds of backpropagation optimization. Therefore, the computational cost is significantly lower than that of BNN-Auto. In other words, CPT-LSM is more computationally lightweight. At the same time, BNN-Auto achieves stronger nonlinear fitting ability, uncertainty quantification ability, and better prediction performance at the cost of higher training cost. The time costs for the above two methods were incurred during the initial draft and the first review period when they were completed at the school workstation. Recently, during the summer vacation, the school server was restricted in its operation and after multiple negotiations, the issue was not resolved. Therefore, we retimed the CPT-LSM and the publicly available version of BNN-Auto using the same data, cross-validation division, and computing resources. Both methods used 535 nuclides and were run sequentially and single-threaded on the same laptop. The test

platform was Windows 10, Intel Core i7-8565U at 1.80 GHz CPU and approximately 16 GB of memory. This ensured that the time for CPT-LSM and BNN-Auto came from exactly the same computing resources. The data file reading time was not included in the model training and prediction time. The average single-fold pure training time of BNN-Auto was 77.67 ± 25.93 seconds, and the average complete process including validation and prediction was 77.80 ± 25.94 seconds. The ten-fold cross-validation required training ten independent models, and the actual total time was 777.997 seconds, approximately 12.97 minutes; this total time was the cost for model robustness assessment, not the necessary single training cost for obtaining a final model. Therefore, the actual training time for a BNN-Auto model was approximately 78 seconds (about 1.3 minutes), which is at the minute level, rather than several hours or even days. After training, 24 Monte Carlo posterior forward propagation was used to predict all 535 nuclides, and the median time for all predictions was only 0.369 seconds. The median time for CPT-LSM to complete one fold parameter fitting and validation prediction was approximately 1.19×10^{-4} seconds, fitting five parameters using all data was approximately 7.64×10^{-5} seconds, and predicting all 535 nuclides was approximately 5.23×10^{-6} seconds. BNN-Auto took longer than CPT-LSM because BNN-Auto required 180 epoch iterations of iterative backward propagation, variational Bayesian parameter learning, and multiple MC posterior predictions, while CPT-LSM only needed to perform linear least squares fitting of five parameters. Due to the absolute time of CPT-LSM being close to the microsecond to millisecond level of the timer, the relative multiplier calculated with it as the denominator would be very large; however, from the actual usage time, BNN-Auto still required approximately 78 seconds for a complete training on an ordinary laptop, and less than 0.4 seconds for predicting all samples, so there was no need for a training burden of several hours or even days. Previously, the total time for ten-fold cross-validation on NVIDIA GeForce RTX 3090 was approximately 436.7 seconds, with an average of approximately 43.6 ± 11.5 seconds per fold; the average complete process for each fold on this laptop CPU was approximately 77.80 ± 25.94 seconds. Therefore, RTX 3090 was still faster in absolute time, but its acceleration ratio relative to the CPU was not as significant as that of large neural networks. The two sets of results are not conflicting: they jointly indicate that BNN-Auto is smaller in scale, and the GPU can further shorten the training time, but even without using the GPU, an ordinary laptop can complete a model training in approximately 1-2 minutes. Based on the above results, we did not include the calculation times of the two methods in the revised manuscript.

Specific comments are detailed below:

1) Page 7. Sentences following: "The encoder takes the 8-dimensional input features and projects them through ..."
The following points require clarification:
a) What specific structure was used in this work? Have the authors used different activation functions for each block (i.e., encoder, decoder, and output blocks)? If so, the structure of each block should be explained clearly.
b) What is the structure of the Output block (which is not mentioned)? For example: $8 \to ? \to ? \to 1$.

Response: Thank you for pointing out that the original manuscript did not provide clear explanations for the specific structure of the network and the activation functions. Based on your suggestion, we have added information about the number of neurons, output dimensions, activation functions, as well as the differences between deterministic layers and Bayesian layers in the revised manuscript. The details are as follows:
a) Our model takes an 8-dimensional input feature x as input. The encoder adopts a structure of $8 \to 128 \to 64 \to 8$. After the first two fully connected layers, Batch Normalization and ELU activation functions are used successively. The final $64 \to 8$ linear layer does not use an additional activation function, and its output is an 8-dimensional latent variable z. The decoder adopts a structure approximately symmetric to the encoder: $8 \to 64 \to 128 \to 8$. The first two hidden layers also use Batch Normalization and ELU activation functions, and the final $128 \to 8$ reconstruction layer does not use an activation function, which is used to reconstruct the standardized continuous input feature. Besides the autoencoder, the model also contains an auxiliary mapping branch that maps the original 8-dimensional input through a $8 \to 8$ linear mapping and then passes through Layer Normalization and ELU activation functions to obtain the auxiliary representation $s(x)$. During the prediction stage, the 8-dimensional latent representation z, the 8-dimensional auxiliary representation $s(x)$, and the original 8-dimensional input x are concatenated to form a 24-dimensional fused representation: $h_{in} = (z, s(x), x)$ in R^{24}. It is worth noting that the 8-dimensional reconstruction result output by the decoder is only used for the autoencoder reconstruction branch and the reconstruction loss calculation, and is not input into the prediction module together with z, $s(x)$, and x. The prediction module uses the above hybrid fused representation.
b) The "Output block" you mentioned is actually the Bayesian prediction module, and its complete structure is $24 \to 128 \to 64 \to 1$. After the first two Bayesian Linear layers, Batch Normalization and ELU activation functions are used. The final $64 \to 1$ Bayesian Linear layer outputs a scalar. This scalar is then transformed into the final $log_{10}P_\alpha$ prediction value through $pred_{logP_\alpha} = -softplus(raw_{output})$, to satisfy the constraint $P_\alpha \leq 1$ which corresponds to $log_{10}P_\alpha \leq 0$; the corresponding P_α can be obtained by $10^{pred_{logP_\alpha}}$.

2) Page 9. "Hence, the ratio of the number of model parameters to the number of training samples is about 1:82.5." The ratio calculations do not appear to match. If there are 481 training samples and $\approx 4.4 \times 10^4$ trainable parameters, then the sentence should read: "Hence, the ratio of training samples to the number of model parameters is about 1:91.5" (calculated as $4.4 \times 10^4/481 = 91.5$; note the inverted order of training samples and model parameters).

Response: Thank you for pointing out the calculation and presentation errors regarding the ratio of parameters to training samples in the original manuscript. Based on the actual network implementation used in the original manuscript, after converting the prediction module into a Bayesian layer, the model has a total of 44,138 trainable parameters. This statistics include the trainable weights and biases of the deterministic network, the trainable affine parameters of BatchNorm/LayerNorm, and the posterior parameters in the Bayesian Linear layer; it does not include the non-training running statistics of BatchNorm. For each of the 481 training samples in each fold, there are $44{,}138/481 = 91.76$. Therefore, the ratio of the number of training samples to the number of model parameters is approximately 1:91.8. If the 4.4×10^4 parameters after rounding are used for calculation, it will result in

approximately 1:91.5 as you pointed out. We have revised the original manuscript.

3) Page 9. "moreover, combined with the variational inference regularization inherent in Bayesian neural networks (via the KL divergence term) and the reconstruction loss constraint of the autoencoder, the model capacity can be effectively controlled. Consequently, the risk of overfitting is significantly mitigated, ensuring that the model possesses strong generalization capability." The intended meaning is not entirely clear. The authors discuss generalization capabilities as an established fact without having demonstrated them for their specific methodology. They only compared the errors between their BNN-Auto method (σ_{post}) and the previous CPT-LSM (σ_{pre}) on page 10, concluding that lower RMS errors imply better generalization. However, this analysis only demonstrates that the BNN-Auto method has lower errors (likely due to higher flexibility) than the previous CPT-LSM method. Generalization capabilities cannot be concluded without further figures and analyses. While an encoder-decoder strategy with BNN might reduce overfitting, this must be proven for this particular database rather than stated as a general truth. The authors should first demonstrate their claims before stating them. Please revise the manuscript accordingly.

Response: Thank you very much for your insightful comment. We fully agree that generalization capability should not be asserted as an established fact before it is properly demonstrated. We have made the necessary revisions to the descriptions on pages 9 and 10 of the original manuscript.

4) Page 12, Table II.
a) While error bounds (superindex and subindex in the sixth column) are important, they are currently difficult to visualize. Consider using $\log_{10}\mathrm{P}_{\alpha}$ instead of P_{α} in columns 5 and 6 to improve readability.
b) Please add a column with the corresponding $\log_{10}\mathrm{T}_{1/2}$ values from the CPT-LSM method for comparison.

Response: Thank you very much for your insightful comments and constructive suggestions. We have carefully considered your feedback and have made the following revisions to the manuscript accordingly:
a) We agree that visualizing the error bounds for P_{α} is clearer when using logarithmic values. Following your suggestion, we have revised the sixth column of Table II to present $log_{10}P_{\alpha}$. The corresponding uncertainties are now expressed in the logarithmic scale, which we believe significantly improves readability and facilitates comparison with other quantities in the table. The updated table has been highlighted in blue for your convenience.
b) We appreciate your recommendation to include a comparison with the CPT-LSM method. Accordingly, we have added a new column in Table II that lists the $\log_{10}\mathrm{T}_{1/2}$ values obtained from the CPT-LSM approach. This addition allows for a direct and comprehensive comparison between the BNN-Auto corrected half-lives, the CPT-LSM results, and the experimental data. Furthermore, we have incorporated a discussion of these LSM results into the relevant section of the manuscript (highlighted in blue) and updated Figure 2 to include the LSM data points for a complete visual comparison.

5) Pages 14 and 15: Discussion regarding Fig 4. It is unclear whether the authors used the raining set, the testing set, or both for Fig. 4. Furthermore, there is no explanation for why

the distributions are non-symmetric or non-Gaussian. The authors should address this.

Response: Thank you for your valuable suggestions. Firstly, we have clarified in the revised manuscript that the residual distributions in Figs. 3 and 4 are constructed from all 535 nuclei, i.e., encompassing both the training and validation sets, to illustrate the global improvement of the method. Secondly, we have added a brief explanation regarding the non-symmetric or non-Gaussian shape of the residual distribution in Fig. 4. This originates from the intrinsic heterogeneity of the dataset (159 even-even, 295 odd-A, and 81 odd-odd nuclei with distinct preformation factor systematics) and is further shaped by the Bayesian KL regularization, which concentrates predictions near experimental values while permitting a heavier tail for outliers.

6) Page 16, Fig. 5: This figure is very clear, but the caption should clarify which results are from the current BNN-Auto work and cite the corresponding references for the other methods. Additionally, the authors should include the CPT-LSM method for consistency.

Response: Thank you very much for your meticulous review and valuable suggestions regarding this article. Regarding the suggestion to clearly distinguish the current work from other methods as indicated in the figure captions, we have improved the captions of Figure 5 in the revised manuscript. We explicitly stated that "BNN-Auto" represents the method proposed in this study, and added the corresponding references for other comparison methods (RF, GP, SFNN, L-LSR). This modification has made the sources of each method more clear and straightforward. Furthermore, regarding the suggestion of incorporating the CPT-LSM method for comparison in Figure 5, we fully understand your considerations. However, after careful deliberation, we believe it is more appropriate not to include CPT-LSM in Figure 5 at present. The reasons are as follows:
1. The core of Figure 5 lies in introducing machine learning methods into the description of the α-particle preformation factor. Therefore, the original design intention of Figure 5 is to focus on the horizontal comparison among different machine learning methods to demonstrate the advantages of BNN-Auto in this field.
2. CPT-LSM belongs to the traditional least squares parameter fitting method, and its methodological basis (local parameter optimization) is fundamentally different from the Bayesian neural network global probability modeling in this study. If it is included in Figure 5, it may blur the comparison logic and theme of the figure.
3. More importantly, we have conducted systematic and comprehensive quantitative and visual comparisons of CPT-LSM and BNN-Auto in Tables I, II, as well as Figures 2, 3, 4. For example, Table I presents the RMS deviation comparison of the two methods on the training set and validation set; Figures 3-4 show the global improvement of the residual distribution. Therefore, it enables us to fully evaluate the advantages of BNN-Auto over CPT-LSM through multiple independent dimensions.
We hope to maintain the effect of Figure 5 as a specialized comparison of machine learning methods to avoid the mixed comparison of different types of methods in the same figure, which may affect readability and the focus of the argument. Of course, if you insist that it is necessary to present the results of CPT-LSM in Figure 5, we are willing to follow your opinion and make corresponding adjustments.

7) Page 22: "For reliability, the predictions from the improved simple model (ISM) Ref. [72]

are taken as a reference." The authors should explicitly state that this information has been added to the final column of the table.

Response: Thank you very much for your valuable suggestion. We have revised the sentence on page 22 to explicitly state that the ISM predictions are given in the last column of Table III as $\log_{10}\mathrm{T}_{1/2}^{\mathrm{ISM}}$.

8) Page 22, Table III.
a) Consider using $log_{10}P_\alpha$ instead of P_α in column 4 to improve data visualization.
b) For better comparison with the literature, I strongly advise adding the transitions from $119 \rightarrow$ Ts in Table III (refer to Table IV in Ref. [50]).
c) It is advisable to compare the results with Ref. [50] more explicitly, as it implements a similar encoder-decoder structure.

Response: Thank you very much for your valuable suggestions. Regarding the issues related to Table III that you pointed out, we have carefully revised and improved each item. The detailed contents are as follows:
a) Regarding the suggestion to replace P_α in the fourth column with $log_{10}P_\alpha$, we fully agree. To enhance the readability and visualization of the data, we have separately listed $P_\alpha^{BNN-Auto}$ and its corresponding $log_{10}P_\alpha^{BNN-Auto}$ in the fourth and fifth columns of the revised manuscript Table III, and have appropriately adjusted the description of the table in the main text.
b) Regarding the α decay transition data for the additional $Z = 119$ isotope chain, we fully agree with your suggestion. We have added relevant data for 14 transitions from $^{285}119 \rightarrow ^{281}$Ts to $^{309}119 \rightarrow ^{305}$Ts in the revised manuscript Table III, including Q_α, angular momentum l, $P_\alpha^{BNN-Auto}$, $log_{10}P_\alpha^{BNN-Auto}$, and the predicted half-life $\log_{10}\mathrm{T}_{1/2}^{\mathrm{Pre}}$, $\log_{10}\mathrm{T}_{1/2}^{\mathrm{ISM}}$, and $\log_{10}\mathrm{T}_{1/2}^{\mathrm{MLP+AE-Pre}}$.
c) Regarding a more explicit comparison with Ref. [50], we are very grateful for your suggestion. Since Ref. [50] also employs a neural network method with an encoder-decoder structure, conducting a systematic comparison with it holds significant reference value. Therefore, we added $\log_{10}\mathrm{T}_{1/2}^{\mathrm{MLP+AE-Pre}}$. to the last column of Table III in the revised manuscript and compared it side by side with our BNN-Auto prediction values. Additionally, in Figure 11, we also marked the corresponding results from Ref. [50] for intuitive comparison.

9) Page 23. Revise the Summary section: The authors should avoid overstating their findings and revise the section to align with the requested modifications.

Response: We sincerely thank the reviewer for the constructive suggestion regarding the Summary section. we have carefully revised the Summary to adopt a more measured tone, aligning it with the actual scope and results of our study.

We have made modifications according to your comments, which are shown in blue color in the updated version. We hope our responses and modifications can clarify your concerns, and would like to thank you again for your precious time on promoting the quality of this work.

Sincerely yours,

Xiao-Yan Zhu on behalf of all the authors.